PROJETO SISTEMA DE QUIZ

DOCUMENTO DE ARQUITETURA E REQUISITOS (AEP 2º SEMESTRE)

Instituição: UniCesumar (Maringá - PR) | **Curso:** Engenharia de Software
Autores: Nicolas Silva e Felipe GabrielII

1. DESCOBERTA E CONCEPÇÃO (O PROBLEMA E A ODS)

Educadores do Ensino Fundamental enfrentam diariamente a crescente dificuldade de reter a atenção dos alunos diante de aulas fundamentadas em modelos puramente expositivos, marcados por leituras passivas e excesso de slides estáticos. Em paralelo, a presença massiva de smartphones e o acesso contínuo a notificações e redes sociais competem diretamente pelo foco dos estudantes. Conforme destacado em estudos sobre o uso da tecnologia na educação, o imediatismo dos conteúdos digitais altera o comportamento dos alunos, tornando-os mais impacientes frente a métodos tradicionais e gerando dispersão, fadiga cognitiva e desengajamento generalizado em sala de aula. Em vez de recorrer a proibições ineficazes, a oportunidade reside em transformar o dispositivo móvel em uma ferramenta ativa de aprendizagem.

Este projeto alinha-se diretamente à **ODS 4 da ONU (Educação de Qualidade)** ao integrar a tecnologia por meio de metodologias ativas e gamificação. Ao viabilizar a projeção dinâmica de quizzes interativos e a submissão de respostas em tempo real pelos próprios dispositivos dos alunos, a plataforma converte a distração passiva em engajamento participativo e colaborativo. A apuração imediata dos resultados proporciona uma gratificação pedagógica instantânea para os estudantes, ao mesmo tempo em que fornece ao professor relatórios diagnósticos em tempo real, permitindo identificar defasagens de aprendizagem e intervir pedagogicamente de forma assertiva.

2. LISTA DE REQUISITOS FUNCIONAIS (ESCOPO)

RF01 - Criação e Regras: O sistema deve permitir que o professor crie, edite e estruture quizzes contendo diferentes tipos de questões (múltipla escolha, verdadeiro ou falso), definindo também um gabarito, tempo limite e pontuação por pergunta.

RF02 - Gerenciamento de Sessão: O sistema deve possuir uma funcionalidade para projetar o quiz em tela cheia na sala de aula, gerando e exibindo um código de acesso (PIN) para o login da turma.

RF03 - Validação e Ranking Gamificado: O sistema deve validar as respostas em tempo real e exibir as pontuações e rankings parciais de cada pergunta para gerar engajamento e reter o interesse da turma.

RF04 - Painel de Resultados: O sistema deve exibir de forma simplificada a quantidade de acertos e erros dos alunos em cada matéria, auxiliando o professor a identificar as defasagens da turma para revisão direcionada aos pontos críticos.

RF05 - Envio de Feedback e Dúvidas: Ao final de cada partida, o sistema deve disponibilizar um espaço na tela para o aluno avaliar a dinâmica e enviar dúvidas sobre a matéria diretamente para a revisão posterior do professor.

3. O PLANEJAMENTO ÁGIL (CRONOGRAMA / BACKLOG)

Sprint / Data	Épico	User Story	Responsável
Sprint 1 (Início do 2° Bi)	Gestão de Quizzes	COMO UM professor EU QUERO criar, editar e estruturar quizzes com diferentes tipos de questões, gabarito e tempo limite PARA QUE eu possa avaliar a turma. *(Ref: RF01)*	Felipe
Sprint 2 (Meio do 2° Bi)	Dinâmica de Sala	COMO UM professor EU QUERO projetar um código de acesso na tela PARA QUE a turma faça login na partida ao vivo na sala de aula. *(Ref: RF02)*	Nicolas
Sprint 3 (Fim do 2° Bi)	Motor de Jogo	COMO UM aluno EU QUERO que o sistema valide minhas respostas em tempo real, mostrando pontuações e rankings PARA QUE gere mais engajamento. *(Ref: RF03)*	Felipe
Sprint 4 (Fim do 2° Bi)	Relatórios	COMO UM professor EU QUERO ver um painel simplificado de resultados com acertos e erros PARA QUE eu possa identificar e revisar as dificuldades. *(Ref: RF04)*	Nicolas
Sprint 4 (Fim do 2° Bi)	Feedbacks	COMO UM aluno EU QUERO um espaço no fim da partida para avaliar a dinâmica e enviar dúvidas PARA QUE o professor revise posteriormente. *(Ref: RF05)*	Ambos

4. JUSTIFICATIVA TÉCNICA E ARQUITETURAL

Para o Back-end do projeto no 2º Bimestre, optou-se pela utilização de **Java com Spring Boot**. A escolha da linguagem apoia-se em sua tipagem forte e na aplicação sólida dos conceitos de Orientação a Objetos. Como o quiz necessita executar regras bem definidas, tais como a correção dinâmica de diferentes formatos de questões, o cálculo de pontuação em tempo real e o gerenciamento das partidas, a POO facilita imensamente essa organização estrutural. O uso de herança e polimorfismo nas perguntas, somado à relação de composição (1:N) entre o Quiz e suas Questões, garante uma estrutura modular, limpa e de fácil testabilidade, atendendo plenamente aos requisitos da Engenharia de Software.

No âmbito da persistência e comunicação, o sistema contará com o banco de dados relacional **PostgreSQL**, integrado por meio de uma API REST (seguindo o padrão MVC). O PostgreSQL assegura a integridade das tabelas e das relações entre Professor, Quiz, Questões, Sessões e Respostas, cumprindo a exigência de não utilizar dados simulados em memória. Por fim, a arquitetura REST desacopla estrategicamente a tela de projeção do professor da interface dos dispositivos móveis dos alunos. Isso permite que as respostas sejam enviadas e processadas de forma leve, evitando a sobrecarga da rede local, o que é fundamental em cenários onde o Wi-Fi escolar apresenta instabilidade.

5. ESTRUTURAÇÃO DO GITHUB E DIAGRAMAS

5.1 Link do Repositório Público

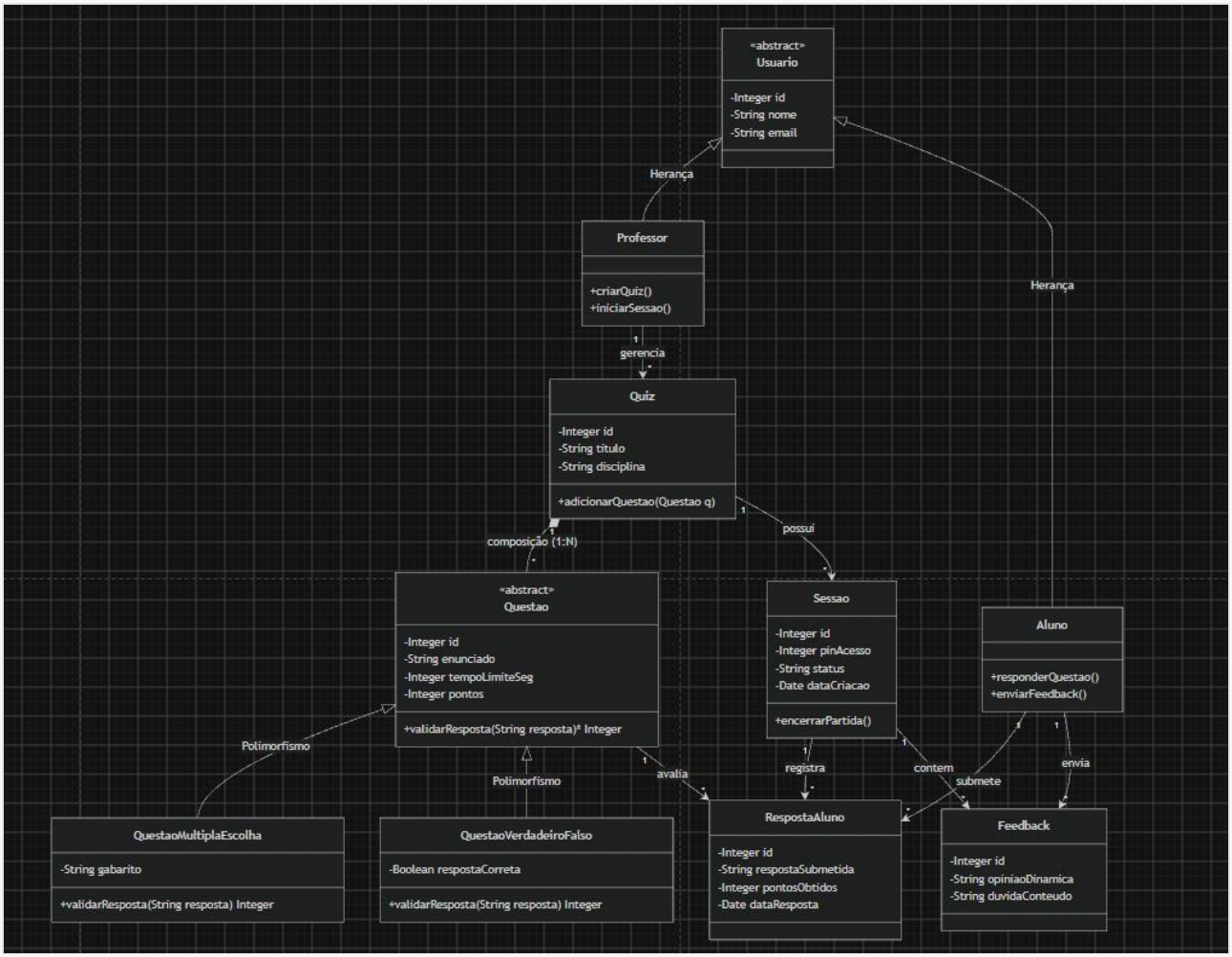
🔗 **Acessar Repositório no GitHub**

https://github.com/FelipeGabrielll/sistema-quiz-atividades-virtuais

5.2 Diagrama de Classes (UML)

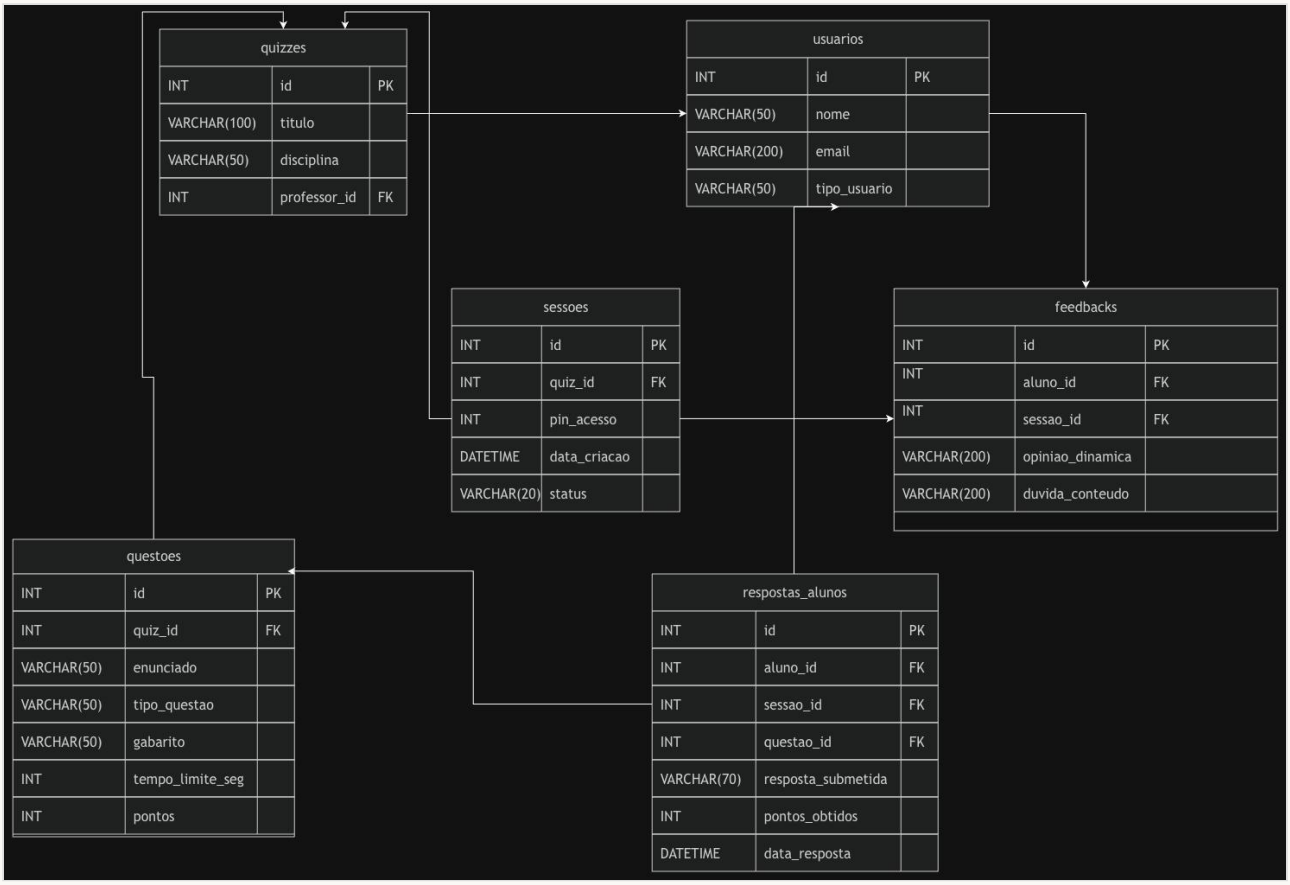
Abaixo, apresenta-se a estrutura de classes que sustenta as regras de negócio do sistema:

- **Herança:** Criação de uma superclasse genérica e abstrata chamada `Usuario`, da qual derivam as subclasses específicas `Professor` e `Aluno`. Há também a superclasse abstrata `Questao`, herdada pelas filhas `QuestaoMultiplaEscolha` e `QuestaoVerdadeiroFalso`.
- **Composição 1:N:** A classe `Quiz` possui uma lista agregada (composição) da classe `Questao`, indicando que um quiz é formado por várias questões. A classe `Sessao` também agrega as instâncias de `RespostaAluno` e `Feedback`.
- **Polimorfismo:** A superclasse `Questao` possui o método de assinatura `validarResposta()`, que é sobrescrito de formas distintas pelas classes filhas, alterando dinamicamente a lógica de pontuação e correção.



5.3 Diagrama de Banco de Dados (DER)

- **Tabela usuarios:** Guarda os logins e os dados de cadastro. Usa a coluna `tipo_usuario` para separar rapidamente quem é professor de quem é aluno.
- **Tabela quizzes:** Salva as informações principais dos questionários. Tem a chave estrangeira `professor_id` para identificar qual professor criou e é o dono daquele jogo.
- **Tabela questoes:** Organiza as perguntas que fazem parte de um quiz (amarradas pela chave `quiz_id`). Também guarda as regras da pergunta, como `tipo_questao`, `gabarito` e `tempo_limite_seg`.
- **Tabela sessoes:** Controla as partidas que o professor abre na sala de aula. Usa o `quiz_id` e guarda os dados daquele jogo ao vivo, como o `pin_acesso` para a turma entrar e o `status`.
- **Tabela respostas_alunos:** Salva cada alternativa enviada pelos alunos. Cruza o aluno, a partida e a pergunta, além de registrar a resposta, os pontos e a hora exata.
- **Tabela feedbacks:** Armazena as avaliações e dúvidas enviadas pela turma no final da partida. Utiliza as chaves `aluno_id` e `sessao_id` para identificar o remetente.


quizzes
INT id PK
VARCHAR(100) titulo
VARCHAR(50) disciplina
INT professor_id FK
usuarios
INT id PK
VARCHAR(50) nome
VARCHAR(200) email
VARCHAR(50) tipo_usuario
sessoes
INT id PK
INT quiz_id FK
INT pin_acesso
DATETIME data_criacao
VARCHAR(20) status
feedbacks
INT id PK
INT aluno_id FK
INT sessao_id FK
VARCHAR(200) opiniao_dinamica
VARCHAR(200) duvida_conteudo
questoes
INT id PK
INT quiz_id FK
VARCHAR(50) enunciado
VARCHAR(50) tipo_questao
VARCHAR(50) gabarito
INT tempo_limite_seg
INT pontos
respostas_alunos
INT id PK
INT aluno_id FK
INT sessao_id FK
INT questao_id FK
VARCHAR(70) resposta_submetida
INT pontos_obtidos
DATETIME data_resposta